Image Processing, Privacy Blur and Face Detection Using OpenCV

Project Title: Image Processing, Privacy Blur and Face Detection Using OpenCV

COURSE/SECTION: CSC 126- BJY11

MEMBERS: TANILON, CALIZO, BACO
Programming Language: Python
Library Used: OpenCV
Development Environment: PyCharm

1. Project Objectives

The main objective of this project is to create a simple image processing program using Python and OpenCV. The program allows the user to select a photo from their computer and perform different image processing operations.

The specific objectives are:

- To load and resize images without changing their original proportions.
- To convert colored images into grayscale and reduce noise using Gaussian blur.
- To detect human faces using the Haar Cascade classifier.
- To apply blur effects on detected faces to help protect privacy.
- To draw rectangles and labels around detected faces and display the total number of faces.
- To save the processed images in a separate folder for easier access.

2. Problem Description and Use-Case

Images are commonly used in school activities, presentations, reports, and other digital works. However, processing images manually can take time, especially when the user needs to resize a photo, detect faces, or blur faces for privacy purposes.

This project helps solve these tasks by combining different image processing techniques into one program. The user can select a photo from their computer using a file selection window. After selecting the image, the program processes

it and produces different outputs, including a grayscale image, a blurred-face image, and a face detection image with labels and face count.

This program is useful for students who wants to learn the basic concepts of computer vision and image processing. It also shows how OpenCV can be used to analyze images and perform simple automated tasks. The results are saved automatically inside a folder called “Modified_output”.

3. Techniques Used

3.1 Image Loading and Resizing

The program uses cv2.imread() to load the selected photo. It checks if the image was loaded successfully before continuing with the next operations.

The program also resizes images that are too large using cv2.resize(). The maximum width is 600 pixels and the maximum height is 300 pixels. This helps the image fit better on the screen while keeping its original proportions.

3.2 Grayscale Conversion

The function cv2.cvtColor() is used to convert the colored image into grayscale. The program uses cv2.COLOR_BGR2GRAY to change the image into different shades of gray.

This process is useful because the Haar Cascade face detector works with grayscale images. It also removes color information that is not needed for this face detection process.

3.3 Gaussian Blur

The program uses cv2.GaussianBlur() to reduce noise and smooth the image. A small kernel size of (3, 3) is applied to the grayscale image before detecting faces.

For the face blur effect, a larger kernel size of (51, 51) is used. This creates a stronger blur on the detected face regions to help hide facial details.

3.4 Haar Cascade Face Detection

The program uses the Haar Cascade classifier to detect faces in the image. The file haarcascade_frontalface_default.xml contains the pre-trained model needed for this process.

The function detectMultiScale() searches for possible faces in the grayscale image. It uses parameters like scaleFactor, minNeighbors, and minSize to control how the detection works.

After detecting the faces, the program stores their positions and sizes. These information are used for drawing rectangles, applying blur effects, and counting the detected faces.

3.5 Face Annotation and Counting

The program uses cv2.rectangle() to draw green rectangles around detected faces. It also uses cv2.putText() to display labels such as Face 1 and Face 2.

The len() function is used to count the total number of detected faces. The result is displayed on the image using red text so the user can easily see how many faces was detected.

3.6 Saving and Displaying Results

The program creates a folder named saved_images using os.makedirs(). If the folder already exists, the program will continue without creating an error.

The function cv2.imwrite() saves the processed images into the folder. These includes the grayscale image, blurred-face image, and annotated face-detection image.

The program also uses cv2.imshow() to display the original image and the processed outputs in separate windows. The user can view the results and press a key to close the windows.

4. Screenshots of Results

Figure 1. Original Image



Description: A raw group photo used in this project

Figure 2. Grayscale Image



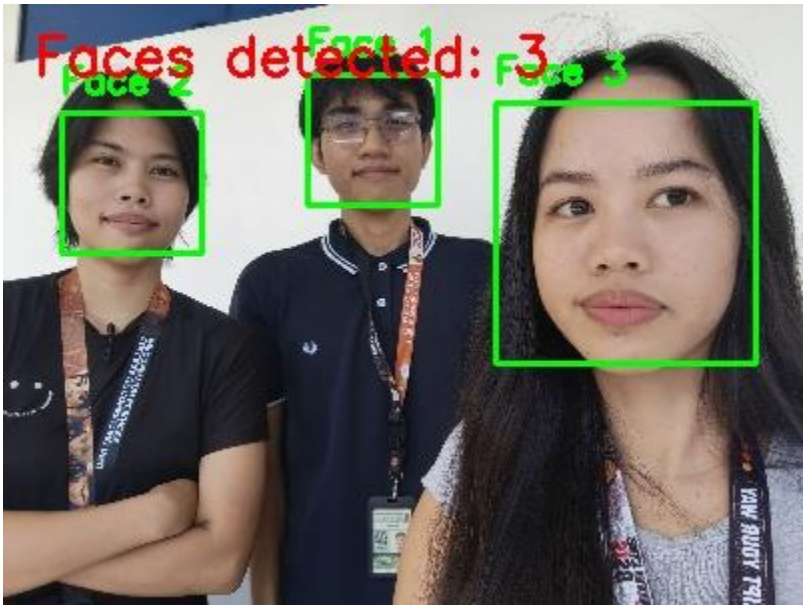
Description: The image is converted into grayscale and Gaussian blur is applied to reduce noise before face detection.

Figure 3. Head Blur Effect



Description: The program applies a strong blur to detected face regions. This helps hide facial details, but it does not guarantee complete privacy.

Figure 4. Face Detection and Counting



Description: Green rectangles are drawn around detected faces, and each face is given a label. The total number of detected faces is also displayed on the image.

5. Accuracy, Behavior, and Limitations

The program can detect faces when the photo contains clearly visible and front-facing people. The grayscale conversion and Gaussian blur helps prepare the image for face detection, while the annotation feature makes the result easier to understand.

However, the accuracy depends on the quality of the selected image. Faces that are turned sideways, covered, too small, or poorly lit might not be detected correctly. Sometimes the program can also detect other objects as faces. Because of this, the displayed face count may not always match the actual number of people in the photo.

The blur effect only works on the regions detected by the face detector. If a face is missed, that face will not be blurred. The program also requires the Haar Cascade XML file to be available in the project folder. Without this file, the face detection process will not work properly.

Another limitation is that the program only provides basic face detection and image processing. It does not recognize the identity of a person or guarantee full privacy protection.

6. Conclusion

This project demonstrates how Python and OpenCV can be used for image processing and face detection. It allows the user to select a photo, resize it, convert it into grayscale, detect faces, apply blur effects, and save the processed images in a separate folder.

Through this project, I learned how to use different OpenCV functions and understand how they work together in one program. I also learned how to use Haar Cascade for face detection and how to manipulate specific parts of an image.